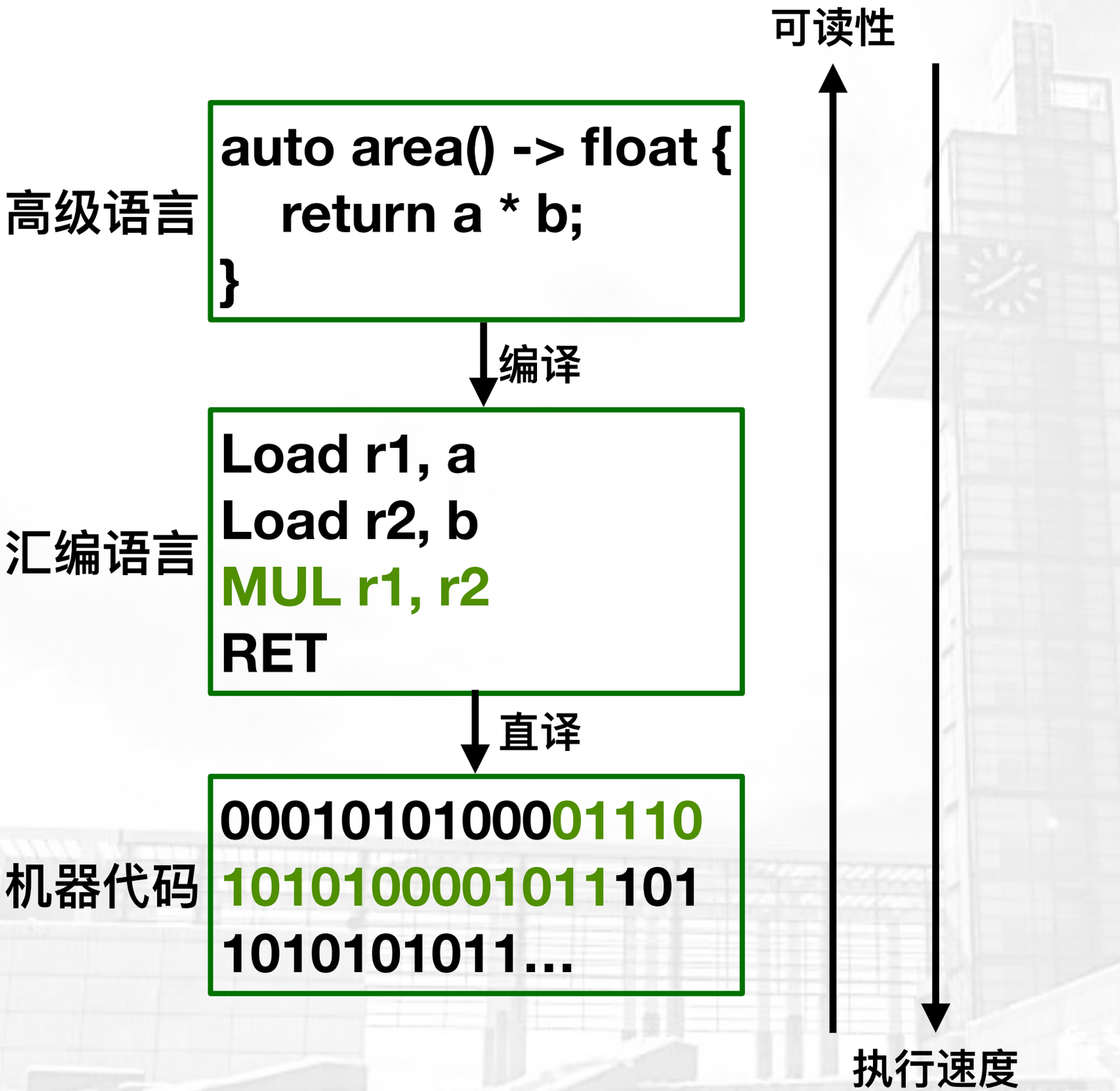
计算机组成与结构
—指令系统1

李瑞　　教授
蒋志平　副教授
计算机科学与技术学院

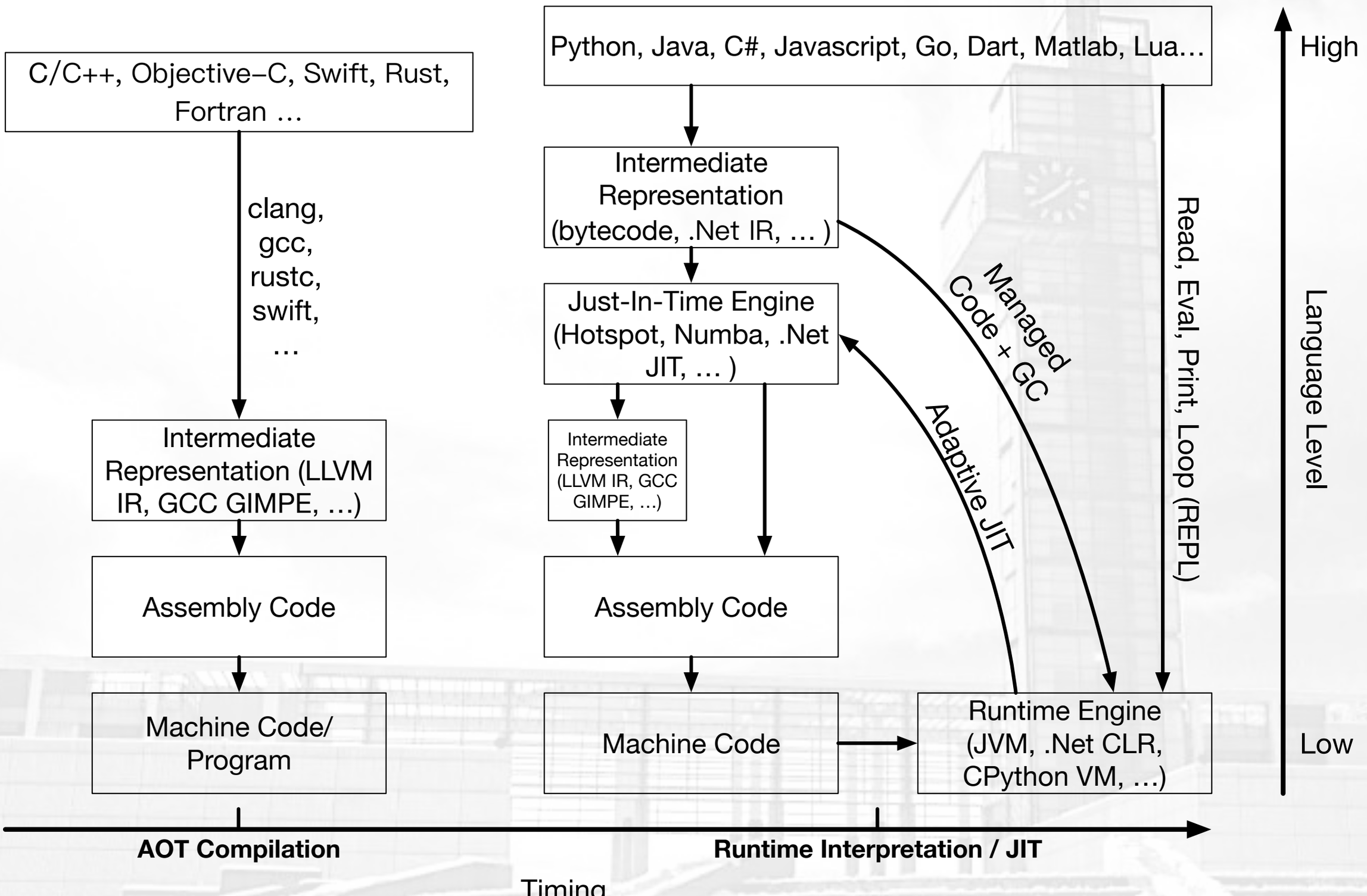
指令系统序言

Code, 2 Code


可读性
高级语言
auto area() -> float {
return a * b;
}
编译
汇编语言
Load r1, a
Load r2, b
MUL r1, r2
RET
直译
机器代码
00010101000001110
10101000010111101
1010101011...
执行速度

Code, 2 Code

Your Code to Machine Code



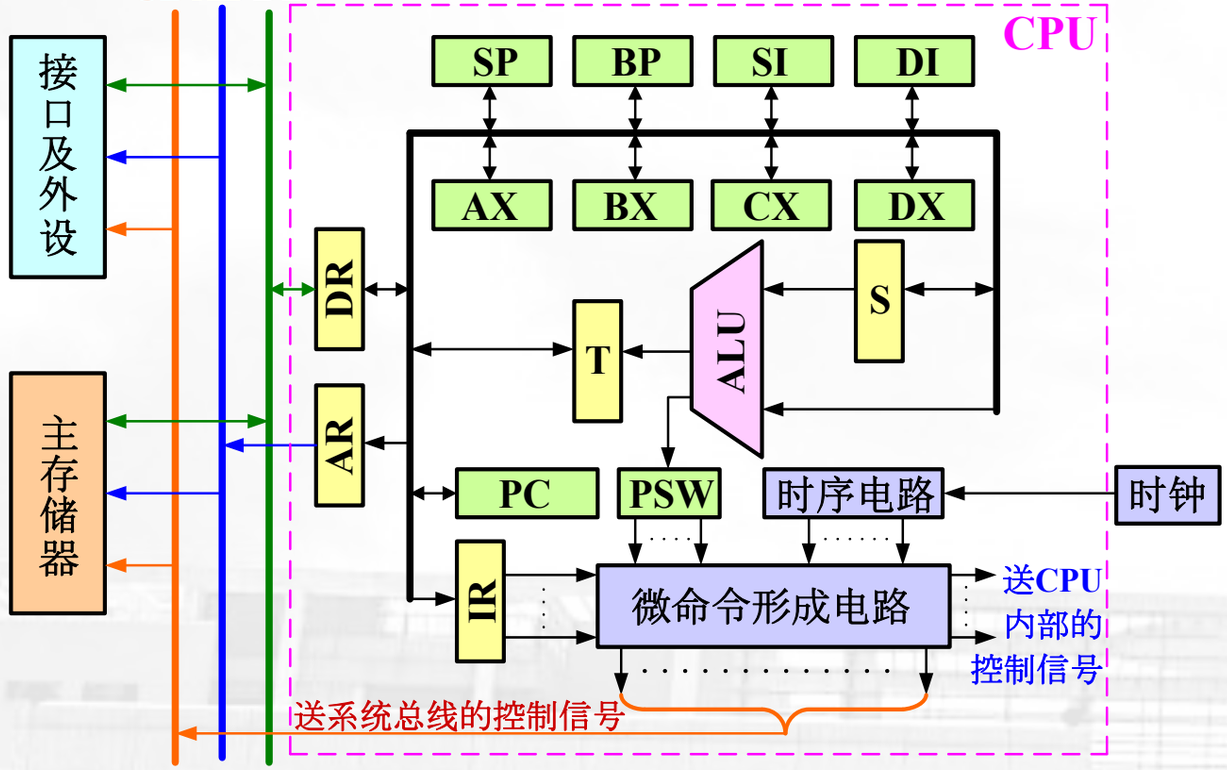
本章内容简介

操作本身，作为指令，存放在内存

高级语言

```
auto area() -> float {
    return a * b;
}
```

一般情况下a和b也在内存



试分析一下，为了执行乘法，至少需要哪些原子操作?

1. 需要从PC所指的内存中，取出当前指令
2. 识别这个指令，发现它有两个操作数
3. 从内存&a地址取a，放在ALU的输入寄存器S；
4. 从内存&b地址取b，放在一个靠近ALU的某个临时寄存器，如AX;
5. 把AX的值送入ALU
6. ALU计算乘法，结果存在临时寄存器T；
7. 把T赋值给其它寄存器
8. 结束函数/子程序，返回

本章内容简介

试分析一下，为了执行乘法，至少需要哪些原子操作?

1. 需要从PC所指的内存中，取出当前指令
2. 识别这个指令，发现它有两个操作数
3. 从内存&a地址取a，放在ALU的输入寄存器S；
4. 从内存&b地址取b，放在一个靠近ALU的某个临时寄存器，如AX;
5. 把AX的值送入ALU
6. ALU计算乘法，结果存在临时寄存器T；
7. 把T赋值给其它寄存器
8. 结束函数/子程序，返回

- 4.1 机器指令的格式与编码

本章内容简介

试分析一下，为了执行乘法，至少需要哪些原子操作?

1. 需要从PC所指的内存中，取出当前指令
2. 识别这个指令，发现它有两个操作数
3. 从内存&a地址取a，放在ALU的输入寄存器S；
4. 从内存&b地址取b，放在一个靠近ALU的某个临时寄存器，如AX;
5. 把AX的值送入ALU
6. ALU计算乘法，结果存在临时寄存器T；
7. 把T赋值给其它寄存器
8. 结束函数/子程序，返回

- 4.1 机器指令的格式与编码
- 4.2 操作数的取值/取址

本章内容简介

试分析一下，为了执行乘法，至少需要哪些原子操作?

1. 需要从PC所指的内存中，取出当前指令
2. 识别这个指令，发现它有两个操作数
3. 从内存&a地址取a，放在ALU的输入寄存器S;
4. 从内存&b地址取b，放在一个靠近ALU的某个临时寄存器，如AX;
5. 把AX的值送入ALU
6. ALU计算乘法，结果存在临时寄存器T;
7. 把T赋值给其它寄存器
8. 结束函数/子程序，返回

- 4.1 机器指令的格式与编码
- 4.2 操作数的取值/取址
- 4.3 8086系统的指令系统

本章内容简介

试分析一下，为了执行乘法，至少需要哪些原子操作?

1. 需要从PC所指的内存中，取出当前指令
2. 识别这个指令，发现它有两个操作数
3. 从内存&a地址取a，放在ALU的输入寄存器S;
4. 从内存&b地址取b，放在一个靠近ALU的某个临时寄存器，如AX;
5. 把AX的值送入ALU
6. ALU计算乘法，结果存在临时寄存器T;
7. 把T赋值给其它寄存器
8. 结束函数/子程序，返回

- 4.1 机器指令的格式与编码
- 4.2 操作数的取值/取址
- 4.3 8086系统的指令系统
- 4.4 汇编语言程序设计

本章内容简介

试分析一下，为了执行乘法，至少需要哪些原子操作?

1. 需要从PC所指的内存中，取出当前指令
2. 识别这个指令，发现它有两个操作数
3. 从内存&a地址取a，放在一个靠近ALU的某个临时寄存器，如AX;
4. 从内存&b地址取b，放在ALU的输入寄存器S;
5. 把AX的值送入ALU
6. ALU计算乘法，结果存在临时寄存器T;
7. 把T赋值给其它寄存器
8. 结束函数/子程序，返回

本章所有内容：

- 4.1 机器指令的格式与编码
- 4.2 操作数的取值/取址
- 4.3 8086系统的指令系统
- 4.4 汇编语言程序设计
- 4.5 RISC/CISC科普

本章内容简介

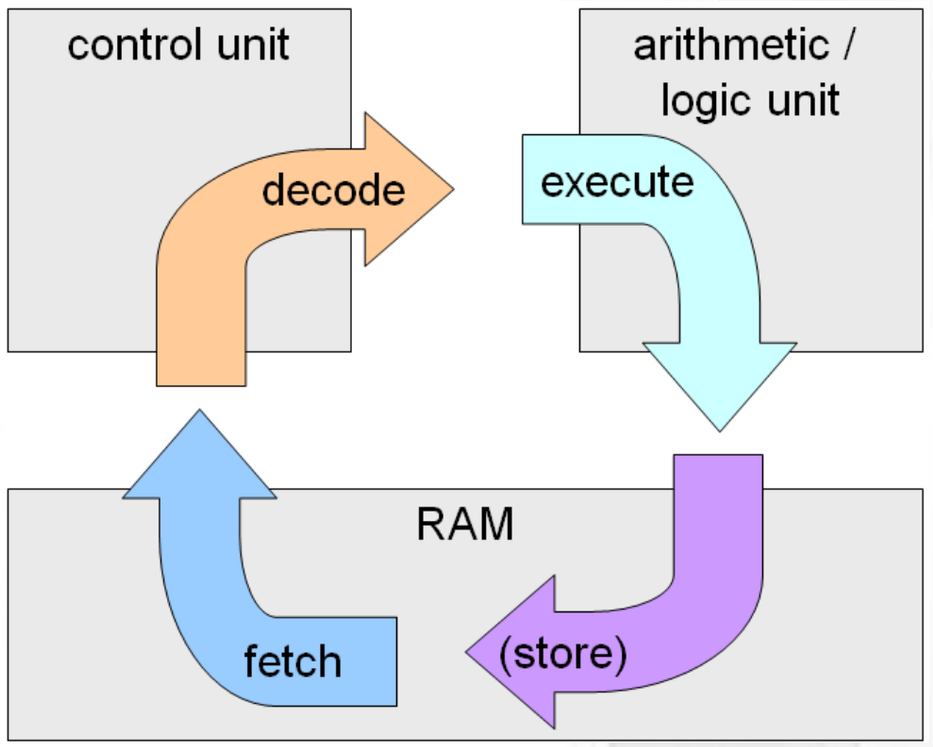
CPU执行指令的3(4)流程循环:

取指令

译码

执行

（存储）



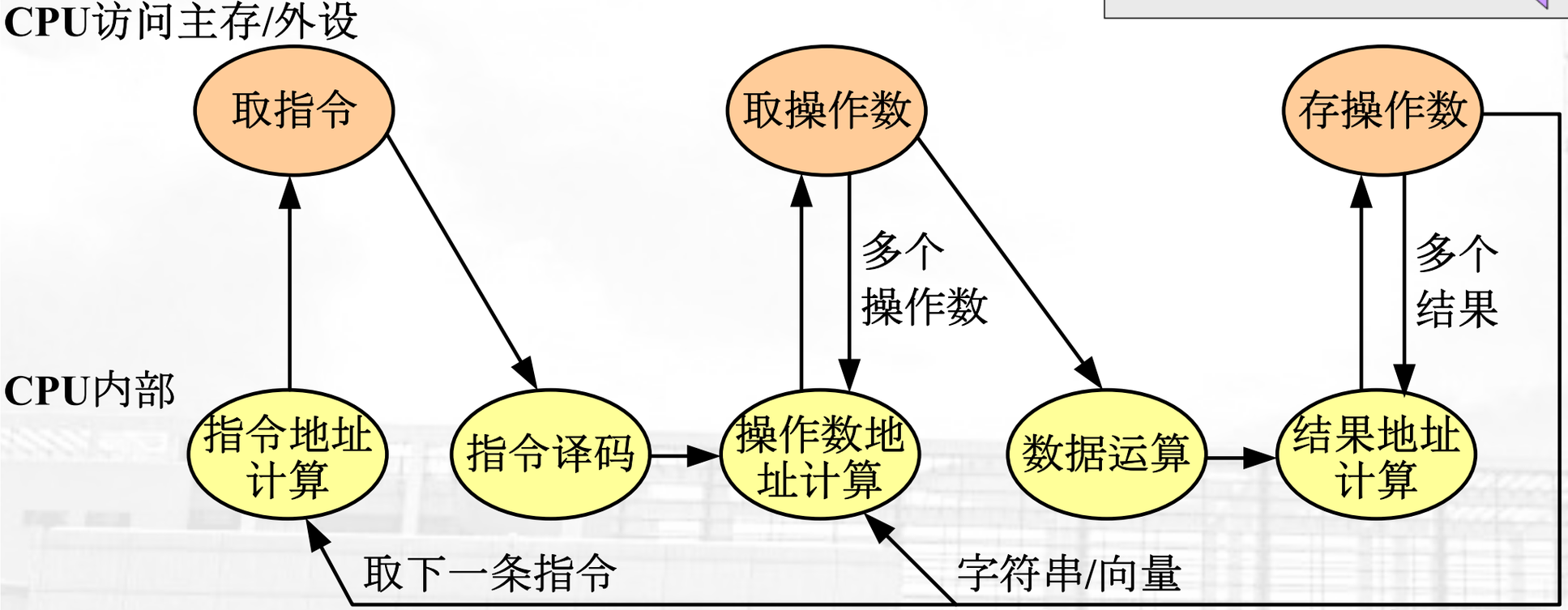


图5.1 机器指令执行状态

指令格式

指令格式

高级语言

```
auto area() -> float {
    return a * b;
}
```

编译

汇编语言

```
Load r1, a
Load r2, b
MUL r1, r2
RET
```

直译

机器代码

```
00010101000 01110
10101000 01011101
1010101011...
```

MUL r1, r2被转译为

这是瞎编的~

“01110101010000 1011”

这一条指令包含以下4条信息：

- MUL指令的编号
- MUL指令能支持两个操作数
- 操作数1指向r1寄存器
- 操作数2指向r2寄存器

指令格式

- 指令 = 操作码Opcode + 操作数 Operand (0~N)
- 操作数，也叫地址码

地址码的内容可以是：地址、数值

地址码的目标可以是：

源/目的操作数，结果，下一条指令

地址（顺序/跳转）

例如，某定长指令格式



指令格式

- 常用的地址码数目：
  - 三地址：ADD Y, A, B ; Y=A+B
  - 双地址：ADD Y, A ; Y=Y+A
  - 单地址：ADD C ; AC=AC+C
  - 零地址

指令格式

例如：计算一个典型的算术表达式：

$$x = \frac{a \times b + c - d}{e + f}$$

用三地址指令编写的程序如下：

```
MUL   X, A, B      ;X←(A)×(B)
ADD   X, X, C      ;X←(X)＋(C)
SUB   X, X, D      ;分子的计算结果在X中
ADD   Y, E, F      ;计算分母,存入Y
DIV   X, X, Y      ;最后结果在X单元中
```

指令格式

$$x = \frac{a \times b + c - d}{e + f}$$

用普通二地址指令编写的程序：

```
MOVE  X, A        ;复制临时变量到X中
MUL   X, B
ADD   X, C
SUB   X, D        ;X中存放分子运算结果
MOVE  Y, E        ;复制临时变量到Y中
ADD   Y, F        ;Y中存放分母运算结果
DIV   X, Y        ;最后结果在X单元中
```

指令格式

$$x = \frac{a \times b + c - d}{e + f}$$

用多寄存器结构的二地址指令编写程序：

```
MOVE  R1, A     ;操作数a取到寄存器R1中
MUL   R1, B
ADD   R1, C
SUB   R1, D     ;R1中存放分子运算结果
MOVE  R2, E
ADD   R2, F     ;R2中存放分母运算结果
DIV   R1, R2    ;最后结果在R1中
MOVE  X, R1     ;最后结果存入X中
```

指令格式

$$x = \frac{a \times b + c - d}{e + f}$$

用一地址指令编写的程序：

```
LOAD    E       ;先计算分母,
                ;取一个操作数到累加器中
ADD     F       ;分母运算结果在累加器中
STORE   X       ;保存分母运算结果到X中
LOAD    A       ;开始计算分子
MUL     B
ADD     C
SUB     D       ;累加器中是分子运算结果
DIV     X       ;最后运算结果在累加器中
STORE   X       ;保存最后运算结果到X中
```

指令格式

用0地址指令编写程序：

```
PUSH   A      ;操作数a压入堆栈
PUSH   B      ;操作数b压入堆栈
MUL           ;栈顶两数相乘，结果压回堆顶
PUSH   C
ADD
PUSH   D
SUB           ;栈顶是分子运算的结果
PUSH   E
PUSH   F
ADD
DIV           ;栈顶是最后运算的结果
POP    X      ;保存最后运算结果
```

$$x = \frac{a \times b + c - d}{e + f}$$

ab*c+d-ef+/

指令格式

不同地址个数指令的特点及适用场合

地址数目	程序的长度	程序存储量	程序执行速度	适用场合
三地址	最短	最大	一般	向量，矩阵运算为主
二地址	较短	很大	很低	一般不宜采用
一地址	较长	较大	较快	连续运算，硬件结构简单
零地址	最长	最小	最低	嵌套，递归，变量较多

指令格式

关于地址码个数结论：

- 对于一般商用处理机，采用多寄存器结构的二地址指令是最理想的。
- 如果强调硬件结构简单，并且以连续运算（如求累加和等）为主，宜采用一地址结构。
- 对于以向量、矩阵运算为主的处理机，最好采用三地址结构。
- 部分RISC处理机也采用三地址指令。
- 对于解决递归问题为主的处理机，宜采用零地址结构。编程容易、节省程序存储量。

指令编码

指令编码

- 指令操作码(Op code)编码方式:
- 编码的考量因素
- 编码方法
  - 定长编码
  - 不定长编码
    - 霍夫曼Huffman Coding
    - 扩展操作码

指令编码

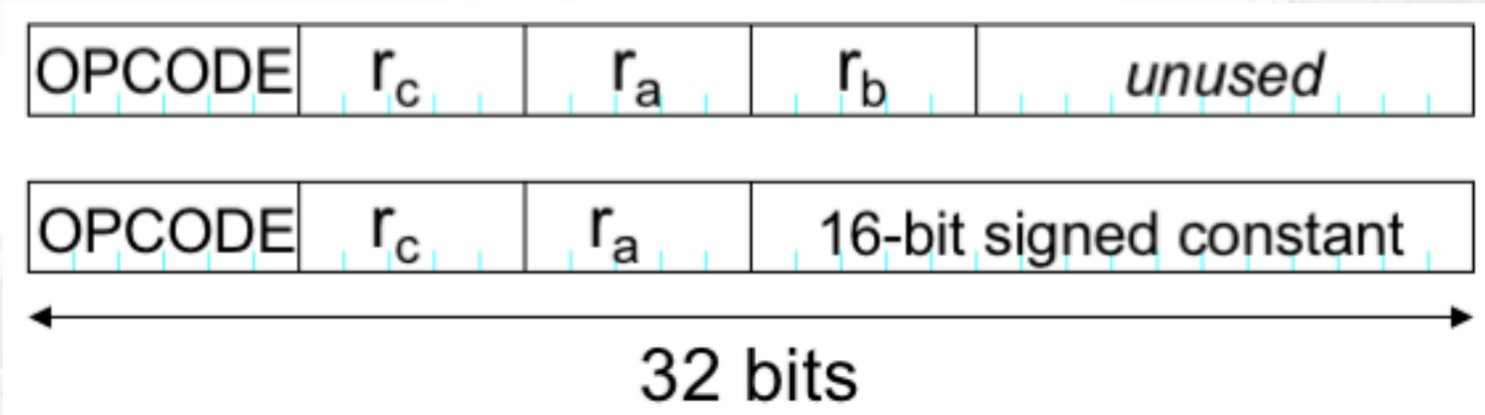
构成一条指令的二进制的位数称为指令长度，一般都是字节的整数倍。

- 内存大小与组织
- CPU数据总线宽度
- CPU内部寄存器的数量
- 寻址方式
- 指令数量

指令编码

定长指令编码：

- 所有指令的编码Opcode长度相同！
  - 不管几操作数的指令，opcode都一样长！
- 显然，会有指令bit浪费，或指令容量受限



指令编码

Huffman指令编码：

- **让高频指令的opcode更短！（不care操作数长度！）**

指令编码

例：假设一台模型计算机共有7种不同的操作码，已知各种操作码在程序中出现的概率如下表，利用固定长度编码法进行操作码编码。

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.45	0.30	0.15	0.05	0.03	0.01	0.01

解：由于N=7 因此，指令操作码固定长度为

$$[lbN]=[lb7]=3$$

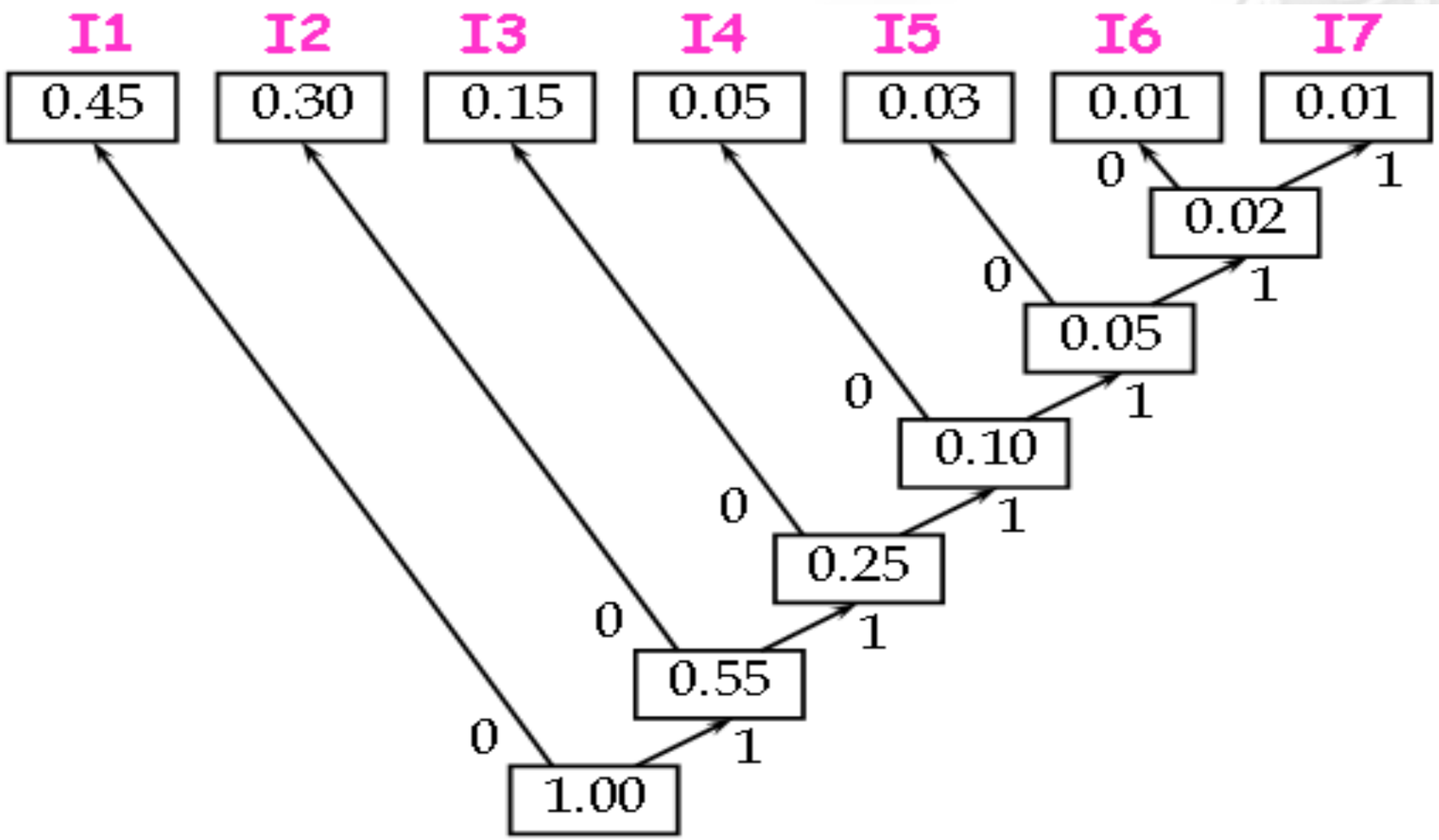
编码结果：

指令序号	概率	编码	操作码长度
I_1	0.45	000	3位
I_2	0.30	001	3位
I_3	0.15	010	3位
I_4	0.05	011	3位
I_5	0.03	100	3位
I_6	0.01	101	3位
I_7	0.01	110	3位

指令编码

例：利用Huffman 编码法进行操作码编码。

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.45	0.30	0.15	0.05	0.03	0.01	0.01



Huffman编码树生成过程

指令编码

编码结果:

指令序号	概率	Huffman编码法	操作码长度
I_1	0.45	0	1位
I_2	0.30	10	2位
I_3	0.15	110	3位
I_4	0.05	1110	4位
I_5	0.03	11110	5位
I_6	0.01	111110	6位
I_7	0.01	111111	6位

采用Huffman编码法的操作码平均长度:

$0.45 \times 1 + 0.30 \times 2 + 0.15 \times 3 + 0.05 \times 4 + 0.03 \times 5 + 0.01 \times 6 + 0.01 \times 6$

$= 1.97$（位）

指令编码

Huffman操作码的主要缺点：

1) 操作码长度很不规整，硬件译码困难
2) 与地址码共同组成固定长的指令比较困难

解决方法之一：　指令操作码的扩展技术

指令编码

扩展指令编码：

- **让操作数多的指令opcode更短，**
- **操作数少的指令opcode最长**

指令编码

原则：短码不能是长码的前缀。

扩展指令编码

15 { 0000 / 0001 / ..... / 1110 }

15 { 1111 0000 / 1111 0001 / ..... / 1111 1110 }

15 { 1111 1111 0000 / 1111 1111 0001 / ..... / 1111 1111 1110 }

15-15-15 编码方法

图5.4

8 { 0000 / 0001 / ..... / 0111 }

64 { 1000 0000 / 1000 0001 / ..... / 1111 0111 }

512 { 1000 1000 0000 / 1000 1000 0001 / ..... / 1111 1111 0111 }

8-64-512 编码方法

指令编码

- 短的操作码与多种地址码配合
- 长的操作码与简单地址码组合
- 指令长度一般设计为总线宽度的整数倍
- 指令的长度为最小可寻址单位的整数倍
- 影响寻址位的因素：
  - 寻址方式的个数
  - 操作数数目
  - 寄存器数目
  - 地址范围
  - 地址可寻址最小单元

指令编码

例：假设某型号的计算机共有14条指令，各条指令的使用频率分别为：0.01、0.15、0.12、0.03、0.02、0.04、0.02、0.04、0.01、0.13、0.15、0.14、0.11、0.03。

试给出定长、只能有两种码长的扩展操作码的两种编码方案，并计算各种方案的平均码长。

【解】

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.15	0.15	0.14	0.13	0.12	0.11	0.04
指令	I_8	I_9	I_{10}	I_{11}	I_{12}	I_{13}	I_{14}
概率	0.04	0.03	0.03	0.02	0.02	0.01	0.01

指令编码

定长编码：

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.15	0.15	0.14	0.13	0.12	0.11	0.04
扩展编码	0000	0001	0010	0011	0100	0101	0110
码长	4	4	4	4	4	4	4
指令	I_8	I_9	I_{10}	I_{11}	I_{12}	I_{13}	I_{14}
概率	0.04	0.03	0.03	0.02	0.02	0.01	0.01
扩展编码	0111	1000	1001	1010	1011	1100	1101
码长	4	4	4	4	4	4	4

其操作码的平均码长：**H＝4**（位）。

指令编码

两种码长的扩展操作码：（方法一）

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.15	0.15	0.14	0.13	0.12	0.11	0.04
扩展编码	000	001	010	011	100	101	110
码长	3	3	3	3	3	3	3
指令	I_8	I_9	I_{10}	I_{11}	I_{12}	I_{13}	I_{14}
概率	0.04	0.03	0.03	0.02	0.02	0.01	0.01
扩展编码	111000	111001	111010	111011	111100	111101	111110
码长	6	6	6	6	6	6	6

其操作码平均码长为：$H=\sum_{i=1}^{n} p_i l_i$

$$H=(0.15+0.15+0.14+0.13+0.12+0.11+0.04)\times 3+$$
$$(0.04+0.03+0.03+0.02+0.02+0.01+0.01)\times 6$$
$$=0.84\times 3+0.16\times 6=3.48\ (\text{位})$$

指令编码

大概率

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.15	0.15	0.14	0.13	0.12	0.11	0.04
扩展编码							
码长							
指令	I_8	I_9	I_{10}	I_{11}	I_{12}	I_{13}	I_{14}
概率	0.04	0.03	0.03	0.02	0.02	0.01	0.01
扩展编码							
码长							

小概率

指令编码

大概率

两种码长的扩展操作码：（方法二）

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.15	0.15	0.14	0.13	0.12	0.11	0.04
扩展编	000	001	010	011	100	101	11000
码长	3	3	3	3	3	3	5
指令	I_8	I_9	I_{10}	I_{11}	I_{12}	I_{13}	I_{14}
概率	0.04	0.03	0.03	0.02	0.02	0.01	0.01
扩展编	11001	11010	11011	11100	11101	11110	11111
码长	5	5	5	5	5	5	5

小概率

其操作码平均码长为：$H=\sum_{i=1}^{n} p_i l_i$

$$H=(0.15+0.15+0.14+0.13+0.12+0.11)\times 3+$$
$$(0.04+0.04+0.03+0.03+0.02+0.02+0.01+0.01)\times 5$$
$$=0.8\times 3+0.2\times 5=3.4\ \text{（位）}$$

指令格式

8086 / 8088指令编码由1～6个字节组成，它包括操作码（第一字节）、寻址方式（第二字节）和操作数（第三到第六字节）三部分组成p

